Routing & API Integration

> Part of the React curriculum. This document covers client-side routing, a typed API client, mirroring backend response shapes, state management beyond local component state, shared component libraries, charting, and build tooling.

1. Routing

react-router-dom provides client-side routing — mapping a URL path to a component, without a full page reload:

```
import { Routes, Route, useNavigate, useParams } from 'react-router-dom';

function App() {
  return (
    <Routes>
      <Route path="/" element={<OverviewPage />} />
      <Route path="/detail/:id" element={<DetailPage />} />
      <Route path="*" element={<NotFoundPage />} />
    </Routes>
  );
}

function DetailPage() {
  const { id } = useParams<{ id: string }>();   // reads the ":id" segment from the matched URL
  return <div>Showing detail for {id}</div>;
}

function SomeButton() {
  const navigate = useNavigate();               // programmatic navigation, e.g. after a form
submits
  return <button onClick={() => navigate('/detail/123')}>Go</button>;
}
```

`useParams` extracts dynamic segments from the URL (`:id` above); `useNavigate` lets you change the current route imperatively (in response to a button click or a completed action), as opposed to a plain `<a>` / `<Link>` element for a route the user clicks directly.

A common pattern is to drive both a navigation menu and this route table from one shared array of `{ path, label, icon }` entries, so adding a new page means adding one entry rather than updating two places that need to stay in sync.

1.1 Nested routes

Routes can nest, letting a parent route render shared layout (a header, a sidebar) around whichever child route currently matches:

```
<Routes>
  <Route path="/" element={<AppShell />}>
    <Route index element={<OverviewPage />} />
    <Route path="detail/:id" element={<DetailPage />} />
  </Route>
</Routes>

function AppShell() {
  return (
    <div>
      <Header />
      <Outlet />   {/* renders whichever nested route currently matches */}
    </div>
  );
}
```

`<Outlet />` is the placeholder where react-router renders the currently-matched nested route's element — this is how a persistent layout (header, navigation) stays mounted while only the "inner" content changes as the user navigates.

2. Talking to a backend API

2.1 A typed API client

Rather than calling `fetch` inline in every component, centralize it in a small client module — one function per endpoint, sharing common request/error-handling logic:

```ts
class ApiError extends Error {
  constructor(public status: number, message: string) {
    super(message);
    this.name = 'ApiError';
  }
}

async function get<T>(path: string): Promise<T> {
  const res = await fetch(`/api${path}`);
  if (!res.ok) {
    const body = await res.json().catch(() => ({ detail: res.statusText }));
    throw new ApiError(res.status, body.detail ?? res.statusText);
  }
  return res.json() as Promise<T>;
}

export const api = {
  overview: () => get<OverviewResponse>('/overview'),
  itemDetail: (id: string) => get<ItemResponse>(`/items/${encodeURIComponent(id)}`),
};
```

A typed `ApiError` (carrying the HTTP status code and the server's own message) lets calling code distinguish error types where it matters, rather than only ever seeing a generic "something went wrong."

2.2 Mirroring the backend's response types

If the frontend is written in TypeScript and the backend defines its response shapes explicitly (e.g., with Pydantic models, as in Python API Backend, Fundamentals §4.2), define matching TypeScript interfaces on the frontend:

```ts
interface OverviewResponse {
  total: number;
  pct_change: number;
  breakdown: Array<{ category: string; value: number | null }>;
}
```

Unless there's a shared-schema code-generation step in place, this mirroring is done **by hand** on both sides — a real, ongoing maintenance cost of a decoupled architecture. Whenever a backend response shape changes, the corresponding frontend type needs the same change, or the two sides will silently drift out of sync — TypeScript will happily let you access a field that no longer exists in the actual JSON, since it only checks against the *declared* type at compile time, not the real response at runtime.

2.3 Query strings and repeated parameters

```
function search(categories: string[], sortBy: string) {
  const params = new URLSearchParams();
  categories.forEach((c) => params.append('category', c));   // repeated key: category=a&category=b
  params.set('sort', sortBy);
  return get(`/search?${params}`);
}
```

3. Error handling patterns for API calls

Beyond the `ApiError` shown in §2.1, distinguish error handling at two levels:

- **Per-request errors** (a single failed fetch) — surfaced via the loading/error/data state pattern from Core Concepts, §5, typically shown inline on the affected page only.
- **Global/unexpected errors** (a bug causing a component to throw during render) — caught by a React **Error Boundary**, a component that catches rendering errors in its children and shows a fallback UI instead of leaving the whole app blank/crashed:

```
import { Component, type ReactNode } from 'react';

class ErrorBoundary extends Component<{ children: ReactNode }, { hasError: boolean }> {
  state = { hasError: false };
  static getDerivedStateFromError() {
    return { hasError: true };
  }
  render() {
    if (this.state.hasError) return <div>Something went wrong.</div>;
    return this.props.children;
  }
}
```

Error boundaries currently must be class components (React doesn't yet provide a hook equivalent) — one of the few remaining places a class component is still the standard pattern in an otherwise fully function-and-hooks codebase.

4. Using a shared component library

Rather than hand-styling every element, many real frontends consume a **shared component library** (sometimes called a design system) — a separately-maintained package of common UI pieces (buttons, page shells, navigation, chart wrappers, form controls) with a consistent visual language. Building against one buys consistency and speed at the cost of a dependency whose release cadence and API you don't fully control — worth being intentional about which parts of your UI use shared components versus custom-built ones. Core

Concepts, §7 covers how a design system is structured (tokens, themes, components, tests) and how to consume one well.

5. State management beyond local state

`useState` (per-component) covers most needs, but two situations call for something more:

5.1 The Context API — for state needed broadly, without prop drilling

Prop drilling is passing a value down through several layers of components that don't themselves use it, just to get it to a deeply-nested component that does. React's Context API avoids this for state that's genuinely global to a section of the tree (a current theme, the logged-in user, application-wide settings):

```
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext<'light' | 'dark'>('light');

function App() {
  const [theme, setTheme] = useState<'light' | 'dark'>('light');
  return (
    <ThemeContext.Provider value={theme}>
      <PageContent />
    </ThemeContext.Provider>
  );
}

function DeeplyNestedComponent() {
  const theme = useContext(ThemeContext);   // reads the value without any prop passed down
explicitly
  return <div className={theme}>...</div>;
}
```

5.2 External state-management libraries

For genuinely complex client-side state (not just data fetched from an API, but substantial derived/interdependent client state), dedicated libraries are common: **Redux** (the longest-established, most structured option, with a formal action/reducer pattern), **Zustand** (a much lighter-weight alternative with a simpler API), and **TanStack Query (React Query)** (specifically for server-state — caching, refetching, and synchronizing data fetched from an API, effectively a more feature-complete alternative to the custom `useAsync` hook from Core Concepts, §5).

Don't reach for these by default. Local component state (`useState`) plus the Context API for the occasional genuinely-global value covers a large fraction of real apps, including most dashboards of the shape this series describes — introduce a dedicated library only once you have a concrete state-management problem (excessive prop drilling across many layers, genuinely complex interdependent client state, or a real need for the

caching/refetching behavior TanStack Query provides) that the simpler built-in tools are demonstrably struggling with.

6. Charting

Rendering interactive charts in a React app is typically done through a wrapper component around a JS charting library (Plotly.js and its React bindings are a common choice for scientific/data-heavy dashboards, given its overlap with the Python ecosystem's own Plotly usage — see Streamlit curriculum, Charting & Visualization for the Python-side equivalent). Whatever library is used, remember accessibility: an interactive chart usually needs an explicit accessible name/description (e.g., an `aria-label` summarizing what the chart shows and its value range) since screen readers can't meaningfully describe an SVG/canvas chart's visual content on their own.

7. Styling approaches

Approach	What it is
Plain CSS / CSS Modules	Ordinary CSS, optionally scoped per-component via CSS Modules (`Button.module.css`) so class names don't collide globally
CSS-in-JS (styled-components, Emotion)	Write CSS directly inside JavaScript/TypeScript, colocated with the component it styles
Utility-first frameworks (Tailwind CSS)	Compose styling from small, single-purpose utility classes directly in markup, rather than writing custom CSS rules
A shared component library's own theming (§4)	If you're building against a design system, styling is often largely handled for you via the library's own theme configuration

None of these is universally "correct" — the right choice depends on team convention, whether you're building against an existing shared component library already, and personal/team preference for colocation vs. separation of markup and styles.

8. Build tooling

Vite is a common modern build tool/dev server for this kind of app — it provides fast local development (a dev server with instant reloads) and produces an optimized production build.

8.1 Dev-time API proxying

In development, the frontend dev server and the backend API usually run as two separate local processes on two different ports. Rather than hard-coding a full URL (`http://localhost:8000/api/...`) into the frontend code — which would then need to change for production — configure the dev server to **proxy** any request under `/api` to the backend's port, so the frontend code can always just call `/api/...` regardless of environment:

```
// vite.config.ts
export default defineConfig({
  server: {
    proxy: {
      '/api': { target: 'http://localhost:8000', changeOrigin: true },
    },
  },
});
```

8.2 Environment variables

Vite exposes build-time environment variables prefixed `VITE_` via `import.meta.env`:

```
const apiBaseUrl = import.meta.env.VITE_API_BASE_URL;
```

Values are baked in at **build time**, not read at runtime — a different value requires a different build, which is a meaningful distinction from a typical backend service that reads environment variables freshly on every process start (see the architecture overview's configuration principle for the general idea; frontend builds are one place where "configure per environment" needs a slightly different mechanism than "read an env var at startup").

8.3 Deploying under a sub-path

If the app will be served at a URL like `example.com/my-app/` rather than the domain root, the build tool needs a configured **base path** so all its generated asset URLs (and client-side route matching) account for that prefix — and if a reverse proxy in front of the app forwards the *full* prefixed path to the app unchanged, the app (and its API, if colocated) needs matching logic on the server side to strip that same prefix before routing, exactly the kind of middleware concern described in Python API Backend, Fundamentals §4.7. Getting these two sides' handling of the same prefix out of sync is a common class of "works locally, breaks in the deployed environment" bug.

8.4 Code splitting

For a larger app, Vite (via Rollup under the hood) supports **code splitting** — loading part of the app's JavaScript only when it's actually needed, rather than one large bundle upfront:

```
import { lazy, Suspense } from 'react';

const DetailPage = lazy(() => import('./DetailPage'));

<Suspense fallback={<Spinner />}>
  <DetailPage />
</Suspense>
```

`lazy()` defers loading a component's code until it's actually rendered; `Suspense` provides a fallback UI to show while that code is being fetched. Worth adopting once a bundle grows large enough that initial page-load time

becomes noticeable — not a default requirement for every small app.

9. Core libraries summary

Library	Used for
react-router-dom	client-side routing
TypeScript	typing the API client and response shapes
Vite (or a comparable bundler)	dev server, build, proxying, environment configuration
A charting library (e.g., Plotly.js)	interactive data visualization
TanStack Query / Zustand / Redux (situational)	more advanced server-state or client-state management, once local state + Context isn't enough

See also

- React curriculum index
- Core Concepts — the component/state/hook fundamentals this builds on
- Testing — verifying all of the above